



8



25



3,750 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

Image Compression

د. مدحت الصوص



RB Informatics; 18/05/2026

نظم الوسائط المتعددة

مقدمة:

يُعد ضغط الصور من التقنيات الأساسية في نظم الوسائط المتعددة، ويهدف إلى تقليل حجم ملفات الصور لتوفير مساحة التخزين وتسريع عملية نقلها، مع المحافظة على جودة الصورة قدر الإمكان. في هذه المحاضرة سنتناول أساسيات ضغط الصور، ونوضح الفرق بين Lossless Compression و Lossy Compression، ثم نتعرف إلى خطوات عمل معيار JPEG، وهو أكثر معايير ضغط الصور استخداماً.

Lossless Compression

يُستخدم Lossless Compression عندما يكون من الضروري استعادة الصورة الأصلية بالكامل بعد فك الضغط، أي دون فقدان أي جزء من بياناتها. وعلى الرغم من إمكانية استخدامه في ضغط الصور، فإنه غالباً لا يحقق نسب ضغط كبيرة، وذلك بسبب طبيعة بيانات الصور التي سنتعرف إليها.

Run-Length Encoding (RLE)

تعد خوارزمية RLE من أبسط خوارزميات الضغط، ويمكن استخدامها في بعض تطبيقات Lossless Compression، كما يمكن الاستفادة من فكرتها في بعض مراحل Lossy Compression. وتعتمد فكرتها على تمثيل البيانات بطريقة تقلل من حجمها عند وجود قيم متكررة.

ولكن بشكل عام، تواجه هذه الخوارزمية مشكلة جوهرية عند تطبيقها على الصور، إذ تعتمد هذه الخوارزمية على وجود تكرار حقيقي للقيم، بينما لا يتوافر هذا التكرار غالباً في الصور الطبيعية، فعلى الرغم من أن العين قد ترى مساحة كبيرة بلون واحد، إلا أن قيم البيكسلات داخل هذه المساحة ليست متطابقة تماماً، بل تختلف بفروق بسيطة من بيكسل إلى آخر. ورغم أن هذه الفروق قد لا تكون ملحوظة بصرياً، فإن خوارزمية الضغط تتعامل مع كل قيمة على أنها مختلفة، وبالتالي لا تستطيع الاستفادة من التكرار بالشكل المطلوب.

For Binary Images

- عند تطبيق RLE على الصور الثنائية، يتم مسح الصورة سطراً سطراً، ثم يُحسب طول كل سلسلة متتالية من القيم المتشابهة، فبدلاً من تخزين كل قيمة على حدة، تُخزن القيمة مع عدد مرات تكرارها، وهذا ما يساعد على تقليل حجم البيانات.

وتنتج هذه الخوارزمية مع الصور الثنائية بشكل جيد، لأن هذه الصور تحتوي غالباً على مناطق متجانسة (مساحات بيضاء أو سوداء كبيرة)، فيتوافر فيها التكرار الحقيقي للقيم الذي تحتاجه RLE.

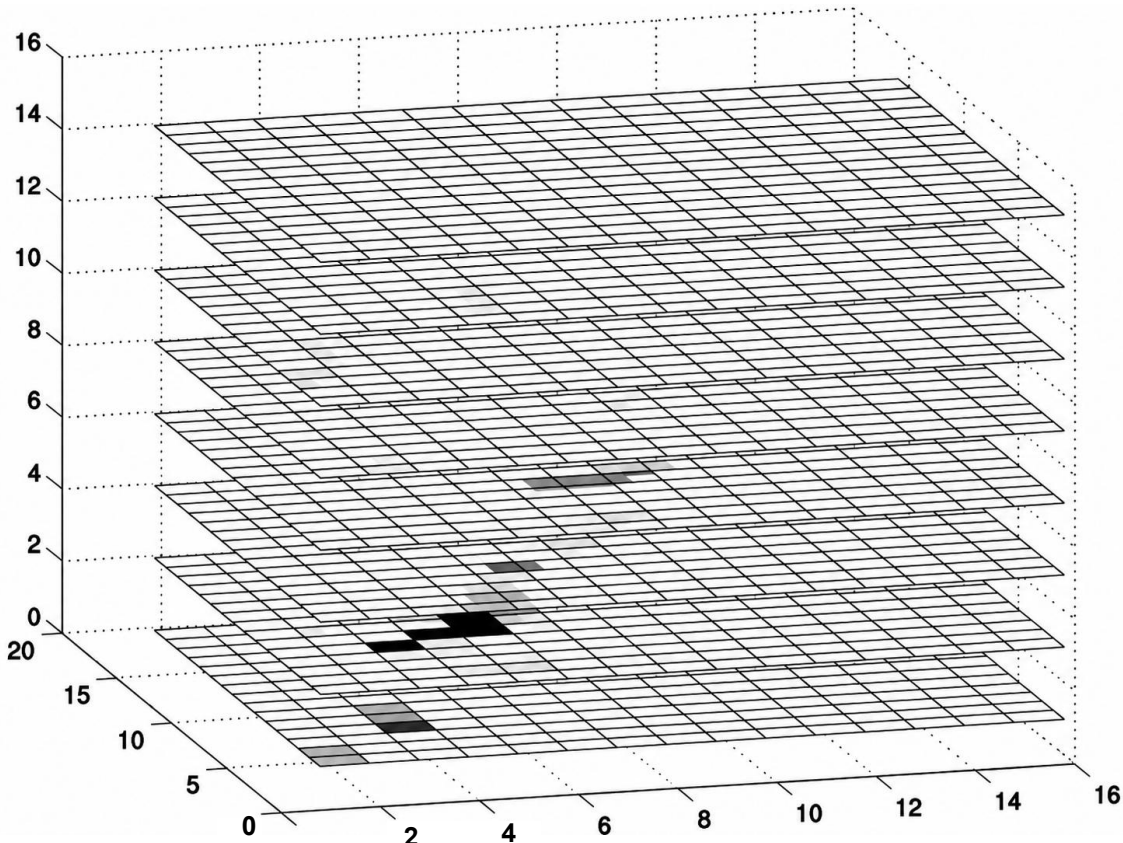
For Grayscale Images

- عند تطبيق RLE على صور Grayscale، تكون نسبة الضغط عادةً محدودة. ويعود السبب إلى ما ذكرناه سابقاً، إذ إن قيم البيكسلات في الصور الطبيعية تتقارب ولكنها لا تتطابق، فلا تتوافر فيها سلاسل القيم المتطابقة التي تحتاجها الخوارزمية.
- ولهذا فإن خوارزمية RLE لا تحقق ضغطاً جيداً إلا إذا احتوت الصورة على مناطق كبيرة تتكون من القيمة نفسها تماماً، مثل مساحة سوداء أو بيضاء متجانسة.
- ولمحاولة تحسين عملية الضغط، طُرحت فكرة تعتمد على تغيير طريقة تمثيل الصورة قبل ضغطها. فبدلاً من التعامل مع قيمة كل بيكسل كاملة، يتم فصل البتات المكونة لكل بيكسل إلى طبقات مستقلة (Bit Planes).
- فإذا كانت الصورة ممثلة باستخدام 8 بت لكل بيكسل، فإنها تتحول إلى ثماني صور ثنائية (Binary Images)، بحيث الصورة الأولى تضم البت الأقل أهمية (LSB) من جميع البيكسلات، والصورة الثانية تضم البت الذي يليه، وهكذا حتى البت الأكثر أهمية (MSB)، وبالتالي نحصل على ثماني صور، أي تمثل كل صورة أحد البتات لجميع البيكسلات. بعد ذلك، تُضغط كل صورة ثنائية بشكل مستقل باستخدام خوارزمية RLE.

مثال:

إذا كانت لدينا 4 بيكسلات بقيم [5, 6, 6, 7] وبالتمثيل الثنائي (101, 110, 110, 111)، فإن الطبقة الأولى (LSB) تكون [1, 0, 0, 1]، والثانية [1, 1, 1, 0]، والثالثة [1, 1, 1, 1].

- يوضح الشكل التالي كيفية تحويل صورة Grayscale إلى مجموعة من الصور الثنائية (Bit Planes)، بحيث تمثل كل طبقة أحد البتات لجميع البيكسلات



كانت الفكرة أن التغير البسيط في قيمة البيكسل يؤدي إلى تغير عدد قليل من البتات، وبذلك تحتوي كل طبقة على سلاسل أطول من الأصفار أو الواحدات، مما قد يساعد على تحسين عملية الضغط.

إلا أن هذه الفكرة واجهت مشكلة عند استخدام Binary Code، لأن التغير البسيط في قيمة البيكسل قد يؤدي أحياناً إلى تغير عدد كبير من البتات. فعلى سبيل المثال، عند الانتقال من العدد 7 إلى 8 يتغير التمثيل الثنائي بالكامل تقريباً، رغم أن الفرق بينهما يساوي واحداً فقط.

ولحل هذه المشكلة، استخدم Gray Code، وهو نظام ترميز يتميز بأن كل رقمين متتاليين يختلفان في بت واحد فقط.

■ يوضح الشكل التالي الفرق بين Binary Code و Gray Code، حيث نلاحظ أن كل رقمين متتاليين في Gray Code يختلفان في بت واحد فقط.

Decimal Number	4 bit Binary Number <u>ABCD</u>	4 bit Gray Code <u>G₁G₂G₃G₄</u>
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1
10	1 0 1 0	1 1 1 1
11	1 0 1 1	1 1 1 0
12	1 1 0 0	1 0 1 0
13	1 1 0 1	1 0 1 1
14	1 1 1 0	1 0 0 1
15	1 1 1 1	1 0 0 0

وعلى الرغم من أن استخدام Gray Code حسن تمثيل البيانات، فإنه لم يحل المشكلة بشكل كامل، لأن أساليب الضغط الإحصائية ما تزال تحتاج إلى تكرار حقيقي للقيم حتى تحقق نسب ضغط مرتفعة، وهذا التكرار لا يتوافر غالباً في الصور الطبيعية.

■ ملاحظة توضيحية:

بالعودة إلى الشكل أعلاه عند الجدول الخاص بـ Gray Code، نلاحظ أن الخط الأحمر يقسم القيم إلى نصفين.

- ففي Binary Code، تكون طريقة التقسيم ثابتة؛ إذ يكون البت الأول في النصف الأول جميعه 0، وفي النصف الثاني جميعه 1. ثم يُقسَّم كل نصف مرة أخرى بالطريقة نفسها، فيصبح البت الثاني 0 ثم 1، ثم يتكرر النمط نفسه في البتات التالية.
- أما في Gray Code، فتختلف طريقة التقسيم فبعد التقسيم الأول إلى 0 و 1، يُعكس ترتيب التقسيم في النصف الثاني ليصبح 1 ثم 0 بدلاً من 0 ثم 1. وعند المستوى التالي من التقسيم يُعكس الترتيب مرة أخرى، وهكذا في كل مستوى، مما يضمن أن كل رقمين متتاليين يختلفان في بت واحد فقط.

■ يمكن أيضاً التحويل من Binary Code إلى Gray Code بخطوات بسيطة:

1. نحتفظ بأول بت من العدد الثنائي كما هو .
2. نحسب كل بت بعده بإجراء عملية XOR بين كل بت والبت الذي يسبقه في العدد الثنائي .

■ مثال (1): تحويل العدد الثنائي 1011 إلى Gray Code



- أول بت هو 1
- $1 \oplus 0 = 1$
- $0 \oplus 1 = 1$
- $1 \oplus 1 = 0$

إذاً:

Binary: 1011
Gray: 1110

■ مثال (2): تحويل العدد 13 إلى Gray Code

نحول العدد 13 أولاً إلى النظام الثنائي:

$$13_{10} = 1101_2$$

ثم نطبق الخطوات السابقة:

- أول بت هو 1
- $1 \oplus 1 = 0$
- $1 \oplus 0 = 1$
- $0 \oplus 1 = 1$

إذاً:

$$13_{10} \rightarrow 1101_2 \rightarrow 1011 \text{ (Gray Code)}$$

«إذا ما اعتدت
إدراك المعالي
أترضى بالتخلف
عنه مرة؟!»

في الامتحان قد يُطلب تحويل عدد مكتوب بالنظام الثنائي مباشرةً إلى Gray Code، أو إعطاء عدد بالنظام العشري.

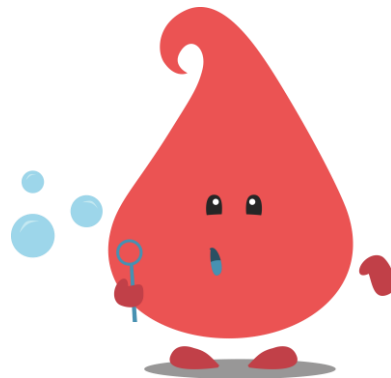
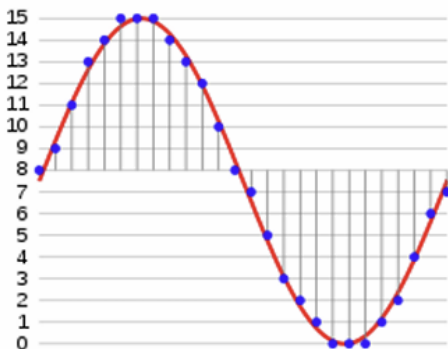
تعتمد الأساليب الإحصائية على فكرة بسيطة، وهي إعطاء تمثيل أقصر للقيم التي تتكرر كثيراً، وتمثيل أطول للقيم التي تتكرر بشكل أقل. لذلك فإنها تحقق أفضل النتائج عندما تكون بعض القيم أكثر تكراراً من غيرها. أما في الصور، وخصوصاً صور Grayscale، فإن معظم قيم البيكسلات تكون متقاربة في عدد مرات ظهورها، كما أن البيكسلات ذات الألوان المتشابهة قد تمتلك قيمة رقمية مختلفة. لذلك لا تستطيع هذه الأساليب الاستفادة من التكرار بالشكل المطلوب، ولهذا لا تعد خياراً مناسباً لضغط الصور بشكل عام.

Lossy Compression

بعد أن تبين أن Lossless Compression لا يحقق عادةً نسب ضغط مرتفعة عند التعامل مع الصور الطبيعية، أصبح الحل هو استخدام Lossy Compression. في هذا النوع من الضغط، لا يكون الهدف استعادة الصورة الأصلية Byte by Byte بعد فك الضغط، وإنما الحصول على صورة تبدو مطابقة أو قريبة جداً من الصورة الأصلية بالنسبة للعين البشرية، حتى وإن اختلفت بعض القيم الرقمية. ولتحقيق ذلك، تحذف بعض البيانات التي يكون تأثيرها البصري محدوداً، مما يسمح بالحصول على نسب ضغط أكبر مقارنةً بأساليب Lossless Compression.

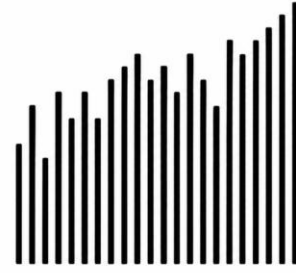
Pulse Code Modulation (PCM)

- قبل دراسة بعض تقنيات ضغط الصور، من المفيد تذكر إحدى الأفكار الأساسية التي سبق استخدامها في ضغط الصوت، لأنها ستستخدم أيضاً عند التعامل مع الصور.
 - تُعد PCM من التقنيات الأساسية المستخدمة في تحويل الإشارات التناظرية (Analog Signals) إلى إشارات رقمية (Digital Signals). وتعتمد هذه التقنية على أخذ عينات من الإشارة (Sampling) ثم تمثيل كل عينة بقيمة رقمية من خلال عملية Quantization.
 - في الإشارات الصوتية تُطبق هذه الفكرة على بعد واحد، أما في الصور فتُطبق على بعدي الصورة (X و Y) لأن الصورة تمثل إشارة ثنائية البعد.
 - Linear PCM (LPCM) هو نوع خاص من PCM، تكون فيه المسافة بين مستويات التكميم متساوية. أي أن الفرق بين أي مستويين متتاليين ثابت دائماً، وعلى الرغم من أن PCM مفهوم عام يشمل عدة أساليب للترميز، فإنه يُستخدم في كثير من الأحيان للدلالة على Linear PCM (LPCM).
- يوضح الشكل التالي فكرة تمثيل الإشارة باستخدام تقنية PCM



Differential Pulse Code Modulation (DPCM)

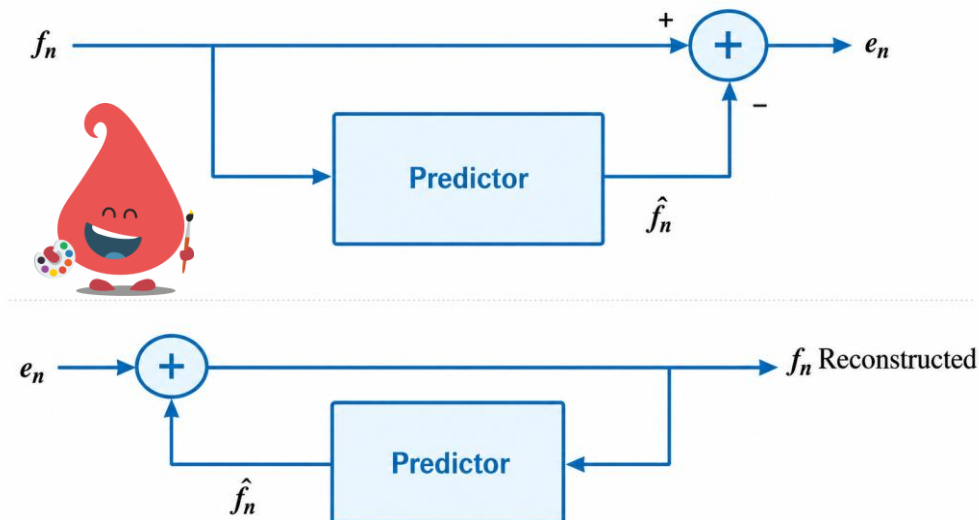
تعد DPCM تطويراً لتقنية PCM، حيث لا تُرسل قيمة كل عينة كما هي، وإنما يُرسل الفرق بين العينات المتتالية. وتستند هذه الفكرة إلى أن العينات أو البيكسلات المتجاورة في الصورة تكون غالباً متقاربة في قيمها، لذلك تكون الفروقات بينها صغيرة وبالتالي تمثل بعدد أقل من البتات، كما أن قيم الفروقات تتكرر بشكل أكبر، وكل ذلك يساعد على تقليل كمية البيانات المطلوب إرسالها.



ويمكن تطوير هذه الفكرة بالاعتماد على التنبؤ (Prediction) كما يوضح الشكل التالي، حيث يعتمد النظام على وجود Predictor يقوم بتوقع قيمة العينة الحالية بالاعتماد على العينات السابقة، ثم يُحسب الفرق بين القيمة الحقيقية والقيمة المتوقعة، ويُرسل هذا الفرق إلى المستقبل بدلاً من إرسال القيمة الأصلية.

مثال:

إذا كانت قيم البيكسلات السابقة هي 100, 105, 103، فإن الـ Predictor قد يتوقع القيمة 104 للبيكسل التالي. فإذا كانت القيمة الحقيقية 102، يُرسل الفرق ($102 - 104 = -2$) بدلاً من القيمة 102. وتعد هذه الطريقة أكثر كفاءة، لأن الفرق بين القيمتين الحقيقية والمتوقعة يكون غالباً صغيراً جداً، مما يجعله يحتاج إلى عدد أقل من البتات لتمثيله. وعند الاستقبال، يُستخدم الفرق المستلم مع القيمة المتوقعة لإعادة بناء قيمة العينة، وبذلك يمكن استرجاع الإشارة الأصلية عندما يكون الفرق المرسل هو الفرق الحقيقي.



Lossy and lossless DPCM

- في Lossless DPCM، يُرسل الفرق الحقيقي بين القيمة الحقيقية والقيمة المتوقعة، لذلك يمكن استعادة البيانات الأصلية بالكامل دون فقدان أي معلومات.
- أما في Lossy DPCM، فيُجرى أولاً تكميم (Quantization) للفرق قبل إرساله، وذلك من خلال تحديد مجموعة من Quantization Levels مثل: 0، 1، 2، 4، 8، 16، ...
- فإذا لم يكن الفرق الحقيقي مساوياً لإحدى هذه القيم، فإنه يُقرب إلى أقرب مستوى منها، ثم يُرسل هذا الفرق بعد التكميم بدلاً من الفرق الحقيقي. ونتيجة لذلك لا تصل القيمة الأصلية تماماً عند فك الضغط، وإنما تصل قيمة قريبة منها، مما يؤدي إلى فقدان جزء من البيانات، ولكنه يحقق نسبة ضغط أعلى.

Transform Coding

- قبل البدء بعمليات ضغط الصور، يمكننا أحياناً تغيير طريقة تمثيل الصورة بدلاً من العمل عليها في فضاءها الأصلي. أي أننا لا نُجري المعالجة مباشرة على قيم البيكسلات، وإنما نقوم بتحويل الصورة إلى فضاء آخر يساعدنا على التعامل معها بطريقة أسهل.
- في هذا القسم تحول الصورة إلى فضاء قريب من Fourier Transform، بحيث تمثل البيانات بطريقة مختلفة تجعل عملية الضغط أكثر كفاءة.
- وبشكل عام، الهدف من هذا التحويل هو نقل الصورة إلى تمثيل جديد يجعل البيانات أكثر تركيزاً وأسهل في الحذف أو التصغير عند الضغط، بدل أن تكون موزعة بشكل عشوائي في فضاء الصورة الأصلي.

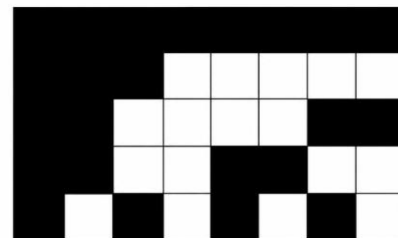
Image Frequencies

- كما أن الإشارات الصوتية تحتوي على ترددات (Frequencies) ناتجة عن اهتزازات الصوت، فإن الصور أيضاً تحتوي على ترددات، إلا أنها ترتبط بالتغيرات في قيم البيكسلات داخل الصورة.
 - وتعد الحواف (Edges) المصدر الرئيسي للترددات العالية (High Frequencies) في الصور، لأنها تمثل انتقالاً مفاجئاً بين لونين أو درجتين سطوع مختلفتين.
 - أما المناطق ذات اللون أو الإضاءة المتجانسة، فتمثل ترددات منخفضة (Low Frequencies)، لأنها لا تحتوي على تغيرات كبيرة في قيم البيكسلات.
- لذلك، عند ضغط الصور لا تعامل جميع أجزاء الصورة بالطريقة نفسها، بل يُحافظ بدرجة أكبر على المعلومات المهمة، بينما يمكن ضغط المعلومات الأقل أهمية بشكل أكبر. ولهذا السبب قد نلاحظ عند حفظ الصورة بصيغة JPEG أن الحواف تصبح أقل حدة قليلاً، لأن جزءاً من البيانات المرتبطة بالترددات العالية يكون قد حُذف أثناء عملية الضغط.



Low Frequencies

High Frequencies



Discrete Cosine Transform (DCT)

- ومن أهم التحويلات المستخدمة لنقل الصورة إلى فضاء آخر هو DCT، والذي يُعد الأساس الذي تعتمد عليه خوارزمية JPEG.
- يشبه DCT في فكرته Fourier Transform، إلا أنه يتميز بأنه أبسط في التطبيق وأكثر ملاءمة لضغط الصور، لذلك يُستخدم عملياً بدلاً من تحويل فورييه في كثير من تطبيقات معالجة الصور.
- يعتمد كل من Fourier Transform و DCT على تمثيل الصورة باستخدام الترددات بدلاً من قيم البيكسلات مباشرة، إلا أن DCT يستخدم دوال Cosine فقط أي أن معاملاته تكون أعداداً حقيقية، بينما يعتمد Fourier Transform على الدوال الأسية المركبة، ولذلك تكون عملياته الحسابية أبسط ويحقق أداءً أفضل في تطبيقات ضغط الصور.
- في DCT تتركز معظم طاقة الصورة في عدد صغير من معاملاته مقارنةً بـ DFT، مما يجعل من الممكن الاحتفاظ بالمعاملات المهمة فقط والتخلص من المعاملات الصغيرة عند الضغط، وهو ما سنوضحه لاحقاً.
- توضح المعادلات التالية تحويل DCT للإشارة أحادية البعد (1-D Signal).



$$F(j) = \sqrt{\frac{2}{N}} C(j) \sum_{i=0}^{N-1} f(i) \cos\left(\frac{\pi j(2i+1)}{2N}\right)$$

$$C(j) = \begin{cases} \frac{1}{\sqrt{2}} & j = 0 \\ 1 & j > 0 \end{cases}$$

$$f(j) = \sqrt{\frac{2}{N}} \sum_{i=0}^{N-1} C(j) F(i) \cos\left(\frac{\pi j(2i+1)}{2N}\right)$$

- حيث إن $F(0)$ تسمى DC coefficient وبقاقي القيم $F(1), F(2), \dots$ تسمى AC coefficients. ويُعد معامل DC الأكبر قيمةً لأنه يمثل مجموع قيم جميع البيكسلات في البوك، بينما تمثل معاملات AC التفاصيل والتغيرات داخل البوك.
- أما عند التعامل مع الصور، والتي تمثل إشارات ثنائية البعد (2-D Signals)، فيستخدم التحويل ثنائي البعد كما هو موضح في المعادلات التالية.

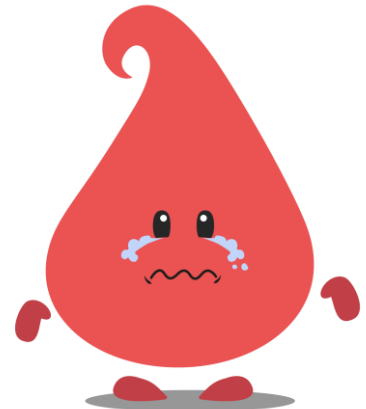
$$F(u, v) = \frac{2c(u)c(v)}{N} \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} f(m, n) \cos\left(\frac{2m+1}{2N} u\pi\right) \cos\left(\frac{2n+1}{2N} v\pi\right)$$

$$u = 0, 1, \dots, N-1 \quad v = 0, 1, \dots, N-1$$

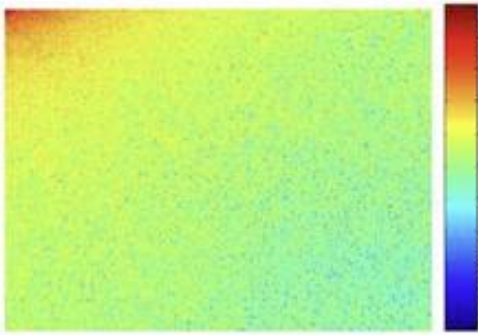
$$C(k) = \begin{cases} \frac{1}{\sqrt{2}} & \text{for } k = 0 \\ 1 & \text{otherwise} \end{cases}$$

$$f(m, n) = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} c(u)c(v) F(u, v) \cos\left(\frac{2m+1}{2N} u\pi\right) \cos\left(\frac{2n+1}{2N} v\pi\right)$$

$$m = 0, 1, \dots, N-1 \quad n = 0, 1, \dots, N-1$$



- تُعد أهم ميزة في DCT هي قدرته على تركيز معظم طاقة الصورة في عدد صغير من المعاملات (Coefficients)، بخلاف DFT، حيث تكون الطاقة موزعة على عدد أكبر من المعاملات.
- لدينا الصورة التالية قمنا بتمثيلها في فضاء DFT وفضاء DCT.



DFT



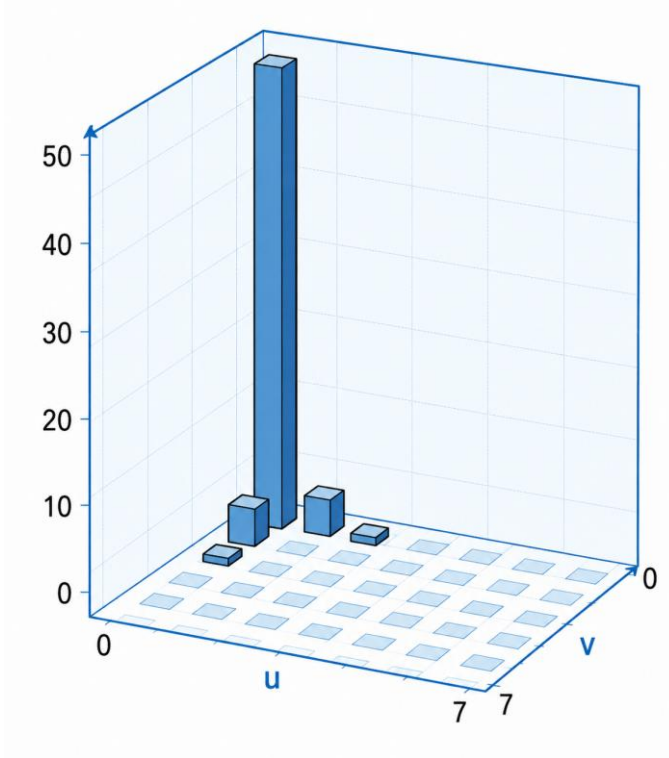
DCT



- يمثل لون كل عنصر قيمة أحد معاملات التحويل، حيث تدل الألوان الباردة على القيم الصغيرة، بينما تمثل الألوان الدافئة القيم الأكبر.

نلاحظ أنه:

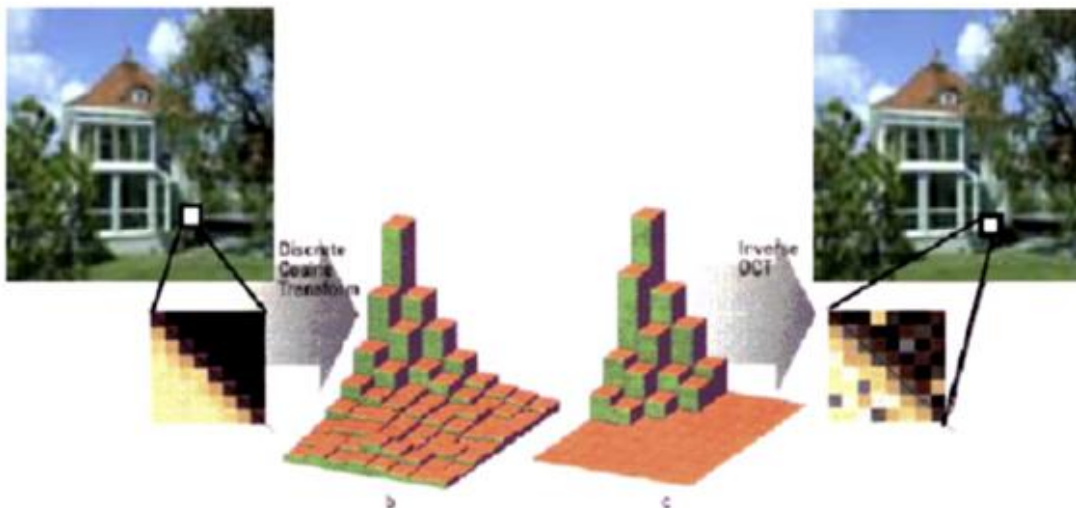
- في DFT القيم الصغيرة موزعة في أماكن مختلفة من المصفوفة (أي أن الإشارة موزعة على قسم كبير من الصورة)، لذلك يصعب التخلص منها دون التأثير على بقية البيانات.
- أما في DCT فتتركز القيم الكبيرة في منطقة صغيرة (أي أن معظم طاقة الصورة تتركز في الزاوية العلوية اليسرى)، بينما تشغل القيم الصغيرة معظم المساحة المتبقية، مما يجعل من الممكن تقريبها إلى الصفر دون أن يتأثر الشكل العام للصورة بشكل كبير.
- يوضح الشكل التالي كيفية توزيع معاملات DCT داخل المصفوفة.



حيث يمثل المعامل الموجود عند $(u=0, v=0)$ معامل DC ، بينما تمثل بقية المعاملات معاملات AC ، تتركز القيم الأكبر في الزاوية العلوية اليسرى عند $(u=0, v=0)$ ، ثم تبدأ القيم بالانخفاض تدريجياً كلما ابتعدنا عن هذه الزاوية. ولهذا السبب يمكن الاحتفاظ بالمعاملات المهمة الموجودة في البداية، بينما يمكن تقريب كثير من المعاملات الصغيرة إلى الصفر.

تجدر الإشارة إلى أن الإحداثيات (u, v) لا تمثل مواقع البيكسلات في الصورة الأصلية (x, y) ، وإنما تمثل مواقع المعاملات في فضاء DCT.

- بعد تحويل الصورة إلى فضاء DCT، يمكن تقليل حجم البيانات من خلال تقريب المعاملات الصغيرة إلى الصفر، مع الاحتفاظ بالمعاملات ذات القيم الكبيرة فقط. والمقصود بحذف القيم هنا ليس حذف أجزاء من الصورة مباشرة، وإنما استبدال معاملات DCT الصغيرة بالقيمة صفر.
- بعد ذلك يُطبق التحويل العكسي Inverse DCT (IDCT) لإعادة الصورة إلى فضاءها الأصلي.





- تكون الصورة الناتجة قريبة جداً من الصورة الأصلية، إلا أنها ليست مطابقة لها تماماً، لأن حذف بعض معاملات DCT يؤدي إلى فقدان جزء من المعلومات، ولذلك تُعد هذه العملية Lossy Compression. ويظهر هذا الفقد غالباً على شكل انخفاض بسيط في حدة الحواف (Edges)، بينما يبقى الشكل العام للصورة قريباً جداً من الأصل.
- وتُعد هذه الفكرة الأساس الذي يعتمد عليه معيار JPEG، حيث تُقسم الصورة إلى كتل (Blocks) صغيرة، يكون حجمها عادة 8×8 بيكسل، ثم يُطبق DCT على كل كتلة بصورة مستقلة، وتُصغر المعاملات الصغيرة، وبعدها يُطبق التحويل العكسي IDCT لإعادة بناء كل كتلة، ثم تُجمع الكتل معاً لإعادة تكوين الصورة.

قد يبدو أن الصورة الناتجة ما زالت تمتلك الأبعاد نفسها وعدد البيكسلات نفسه، لذلك قد يُطرح السؤال: كيف أصبح حجم الملف أصغر؟

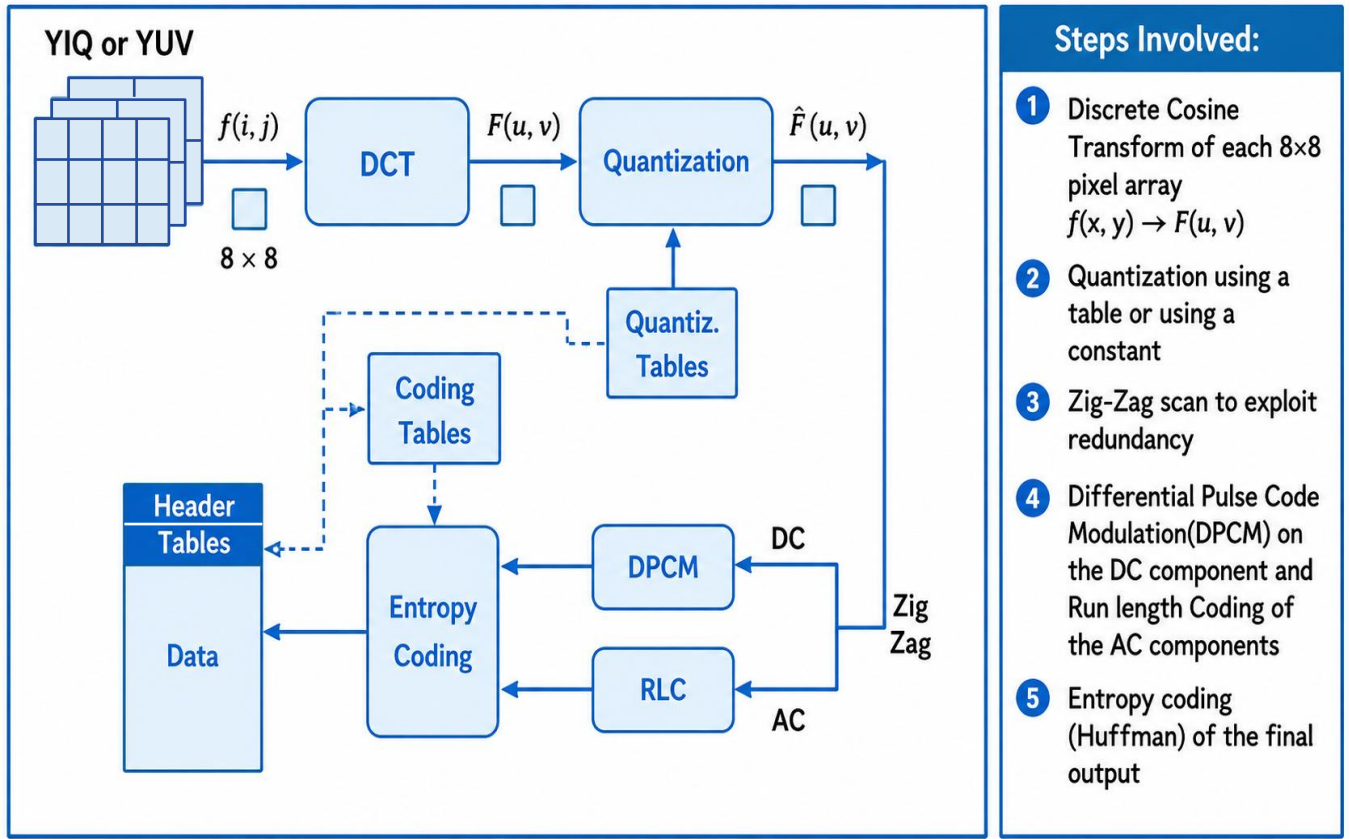
الإجابة هي أن الملف لا يخزن الصورة الناتجة مباشرة، وإنما يخزن معاملات DCT بعد الضغط. وبما أن معظم المعاملات الصغيرة تُستبدل بالقيمة صفر، فيكفي تخزين المعاملات المهمة الموجودة في بداية المصفوفة، بينما يُفترض أن بقية المعاملات تساوي صفرًا عند فك الضغط، ثم يُطبق التحويل العكسي IDCT لإعادة بناء الصورة وعرضها. عادة يطبق الشرح السابق على صور Grayscale، إلا أن المبدأ نفسه يمكن تطبيقه على الصور الملونة، وذلك بمعالجة كل قناة لونية (R و G و B) بصورة مستقلة، ثم تُطبق عليها مراحل الضغط نفسها.

ملاحظة:

لا يُخزن الملف المضغوط الصورة بعد إعادة بنائها، وإنما يخزن معاملات DCT فقط. وعند فتح الملف تُطبق عملية IDCT لإعادة توليد الصورة. لذلك، إذا حُفظت الصورة الناتجة بعد فك الضغط بصيغة BMP، فسيكون حجمها قريباً من حجم الصورة الأصلي، لأن الصورة أصبحت تحتوي مرة أخرى على جميع البيكسلات بأبعادها الأصلية.

JPEG compression

- يُعد (JPEG (Joint Photographic Experts Group أحد أشهر معايير ضغط الصور الثابتة سواء كانت ملونة أو بتدرج الرمادي، ويعتمد بصورة أساسية على Lossy Compression، مع وجود بعض الأنماط التي تدعم Lossless Compression.
- يوجد أكثر من إصدار لهذا المعيار، مثل JPEG 1992 و JPEG 2000، ويكمن الاختلاف الأساسي بينهما في طريقة تنظيم وترميز البيانات داخل الملف.
- ويتميز JPEG بإمكانية التحكم بدرجة الضغط، كما يمكنه في كثير من الحالات تقليل حجم الصورة إلى حوالي $1/10$ أو $1/20$ من حجمها الأصلي مع المحافظة على جودة بصرية جيدة، ويمكن الوصول إلى نسب ضغط أكبر من ذلك، إلا أن زيادة الضغط تؤدي تدريجياً إلى ظهور تشوهات يمكن للمستخدم ملاحظتها.
- تمر عملية الضغط بعدة مراحل متتالية كما يوضح الشكل التالي، وسيتم شرح كل مرحلة منها في الأقسام التالية.



1. Converting to Luminance/Chrominance Color Space

تبدأ خوارزمية JPEG بتحويل الصورة من فضاء الألوان RGB إلى فضاء يفصل بين معلومات الإضاءة (Luminance) ومعلومات اللون (Chrominance)، ويكون ذلك عادةً باستخدام فضاء YUV أو أحد الفضاءات المشابهة مثل YIQ).

في هذا الفضاء:

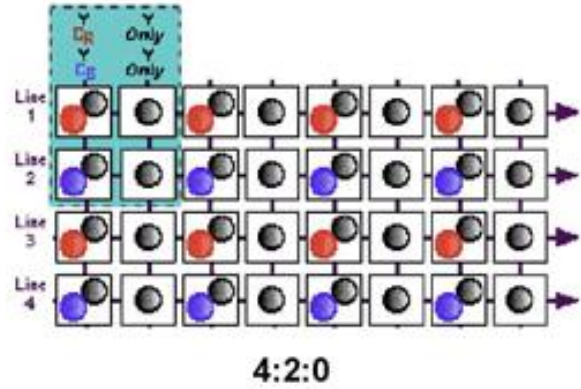
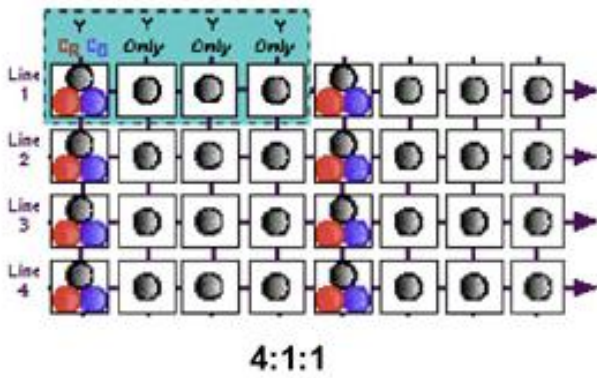
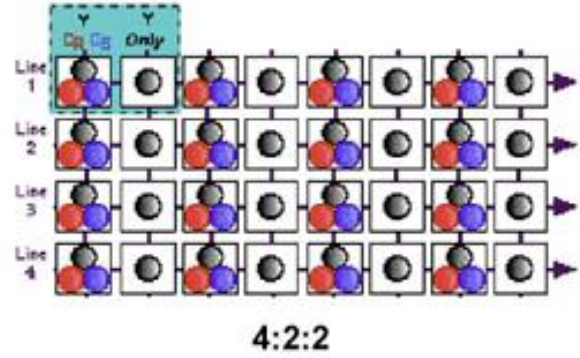
- Y يمثل مكون الإضاءة (Luminance).
- U و V يمثلان مكونات اللون (Chrominance).
- يعتمد JPEG على هذا التحويل لأن العين البشرية أكثر حساسية للتغيرات في الإضاءة من حساسيتها للتغيرات اللونية، لذلك يمكن لاحقاً ضغط مكونات اللون (U و V) بدرجة أكبر، مع المحافظة على مكون الإضاءة (Y) للحفاظ على الجودة البصرية للصورة.
- تعد هذه الخطوة اختيارية في معيار JPEG، ولا تستخدم عند ضغط صور Grayscale، لأنها لا تحتوي على مكونات لونية، وإنما تتكون من مكون الإضاءة فقط.

2. Chrominance Components are Subsampled

- بعد تحويل الصورة إلى فضاء YUV، تأتي المرحلة الثانية من خوارزمية JPEG، وهي Chroma Subsampling.
- تعتمد هذه المرحلة على حقيقة أن العين البشرية أكثر حساسية لتغيرات الإضاءة (Y) من حساسيتها لتغيرات اللون (U و V)، لذلك تحافظ الخوارزمية على جميع قيم الإضاءة، بينما تقلل عدد عينات مكونات اللون، مما يؤدي إلى تقليل حجم البيانات قبل تطبيق بقية مراحل الضغط.

يوضح الشكل التالي أشهر أنظمة Chroma Subsampling.

- 4:4:4 – No subsampling
- 4:2:2, 4:1:1 – horizontally subsample
- 4:2:0 – horizontally and vertically



كل بيكسل في الصورة الأصلية يمتلك ثلاث قيم هي Y و U و V، إلا أنه بعد تطبيق Subsampling لا تُرسل جميع قيم Y و U لكل بيكسل، وإنما تشترك عدة بيكسلات في قيم اللون نفسها، بينما يحتفظ كل بيكسل بقيمة Y الخاصة به.

- 4:4:4 : لا يطبق أي حذف لمعلومات اللون، لذلك يمتلك كل بيكسل قيمه الخاصة من Y و U و V.
- 4:2:2 : تُرسل جميع قيم Y، بينما تُرسل نصف قيم U و V فقط، بحيث يشترك كل بيكسلين متجاورين في قيم اللون نفسها. وعند فك الضغط تُستخدم هذه القيم المشتركة لإعادة بناء البيكسل الذي لم تُخزن قيم اللون الخاصة به .
- 4:1:1 : يشترك كل أربعة بيكسلات في قيمة واحدة من U وقيمة واحدة من V، مما يحقق ضغطاً أكبر على حساب دقة اللون .
- 4:2:0 : لا يعني الرقم 0 عدم إرسال معلومات اللون، وإنما يدل على أن هذا النظام يختلف في طريقة توزيع عينات اللون عن الأنظمة السابقة. ففي هذا النظام تُرسل أيضاً قيمة واحدة من U وقيمة واحدة من V لكل مجموعة من أربعة بيكسلات، لذلك تكون كمية البيانات المرسله قريبة من نظام 4:1:1، إلا أن أسلوب توزيع العينات مختلف.

وبذلك يمكن اعتبار هذه المرحلة نوعاً من Down Sampling لمكونات اللون؛ فبعد تحويل الصورة إلى فضاء YUV تبقى صورة Y بالأبعاد الأصلية، بينما تصبح صور U و V أصغر حجماً. فعلى سبيل المثال، إذا كانت الصورة الأصلية بأبعاد 1000 × 1000، فقد تبقى صورة Y بالأبعاد نفسها، بينما تصبح أبعاد صورتي U و V مثل 500 × 500

500 أو 250×250 بحسب نظام Subsampling المستخدم، مما يؤدي إلى تقليل كمية البيانات المطلوب تخزينها حتى قبل تطبيق بقية مراحل الضغط.

ملاحظة:

- يجب تخزين نظام Chroma Subsampling المستخدم داخل Header الملف، حتى يتمكن برنامج فك الضغط من إعادة بناء الصورة بالطريقة الصحيحة .
- لا تُطبق هذه المرحلة على صور Grayscale، لأنها تحتوي على مكون Y فقط ولا تتضمن مكونات U و V.

3. Create Data Units

- بعد الانتهاء من مرحلة Chroma Subsampling، تُقسم الصورة إلى وحدات بيانات (Data Units) أو كتل (Blocks) صغيرة، يكون حجم كل منها عادة 8×8 بيكسل.
- يُعامل كل بلوك بعد ذلك على أنه وحدة مستقلة، بحيث تُطبق عليه جميع مراحل الضغط اللاحقة بصورة منفصلة.
- وإذا لم تكن أبعاد الصورة من مضاعفات العدد 8، فتكرر الصفوف أو الأعمدة الأخيرة حتى تصبح أبعاد الصورة قابلة للتقسيم إلى بلوكات 8×8 دون ترك أي جزء غير مكتمل .
- يمكن لمعالج JPEG التعامل مع البلوكات بطريقتين:
 - Non-interleaved Mode: تُعالج جميع بلوكات Y أولاً، ثم جميع بلوكات U، ثم جميع بلوكات V.
 - Interleaved Mode: تُعالج بلوكات Y و U و V الخاصة بالمنطقة نفسها من الصورة معاً، ثم يُنقل إلى المنطقة التالية.

4. Apply DCT for Data Units

- بعد تقسيم الصورة إلى بلوكات بحجم 8×8 ، يُطبق DCT على كل بلوك بصورة مستقلة، سواء كان البلوك تابعاً لمكون Y أو U أو V.
- ينتج عن ذلك مصفوفة جديدة أبعادها أيضاً 8×8 ، إلا أنها تحتوي على معاملات DCT بدلاً من قيم البيكسلات.
- وكما أوضحنا سابقاً عند شرح DCT، فإن معظم طاقة البلوك تتركز في الزاوية العلوية اليسرى من المصفوفة، بينما تكون بقية المعاملات صغيرة جداً، مما يجعل المرحلة التالية (Quantization) قادرة على التخلص من كثير من هذه القيم الصغيرة دون التأثير الكبير على جودة الصورة.
- علماً أن تطبيق DCT نفسه قد يسبب فقداناً طفيفاً جداً للبيانات بسبب الدقة المحدودة للحاسوب في حساب دوال Cosine، إلا أن هذا الفقد ضئيل مقارنة بما يحدث في مرحلة Quantization.

5. Quantization

- بعد تطبيق DCT على كل بلوك، تأتي مرحلة Quantization، وهي المرحلة التي يحدث فيها الفقد الحقيقي للبيانات (Lossy Compression) .

- في هذه المرحلة لا تحذف المعاملات الصغيرة مباشرة، وإنما تقسم معاملات DCT على قيم محددة، ثم يُقرب الناتج إلى أقرب عدد صحيح، مما يؤدي إلى تقليل دقة بعض المعاملات.
- يوجد أسلوبان رئيسيان للتكميم:
 - Uniform Quantization: تكون قيمة التكميم $QC(u,v)$ ثابتة لجميع معاملات DCT، أي تُعامل جميع المعاملات بالطريقة نفسها .
 - Non-uniform Quantization: تختلف قيمة $QC(u,v)$ من مكان لآخر داخل المصفوفة، حيث تُستخدم قيم صغيرة في الزاوية العلوية اليسرى لأنها تحتوي على المعاملات الأكثر أهمية، بينما تُستخدم قيم أكبر كلما ابتعدنا عن الزاوية، مما يؤدي إلى تكميم أكبر للمعاملات الأقل أهمية .
- ويعتمد JPEG على Non-uniform Quantization باستخدام Quantization Tables، حيث يوجد عادة جدول خاص بمركبة الإضاءة Y و جدول آخر لمركبتي اللون U و V، وهذه الجداول قياسية (Standard) ومعروفة، ويُخزن النوع المستخدم منها في Header ملف JPEG حتى يتمكن برنامج فك الضغط من استخدام الجدول المناسب.

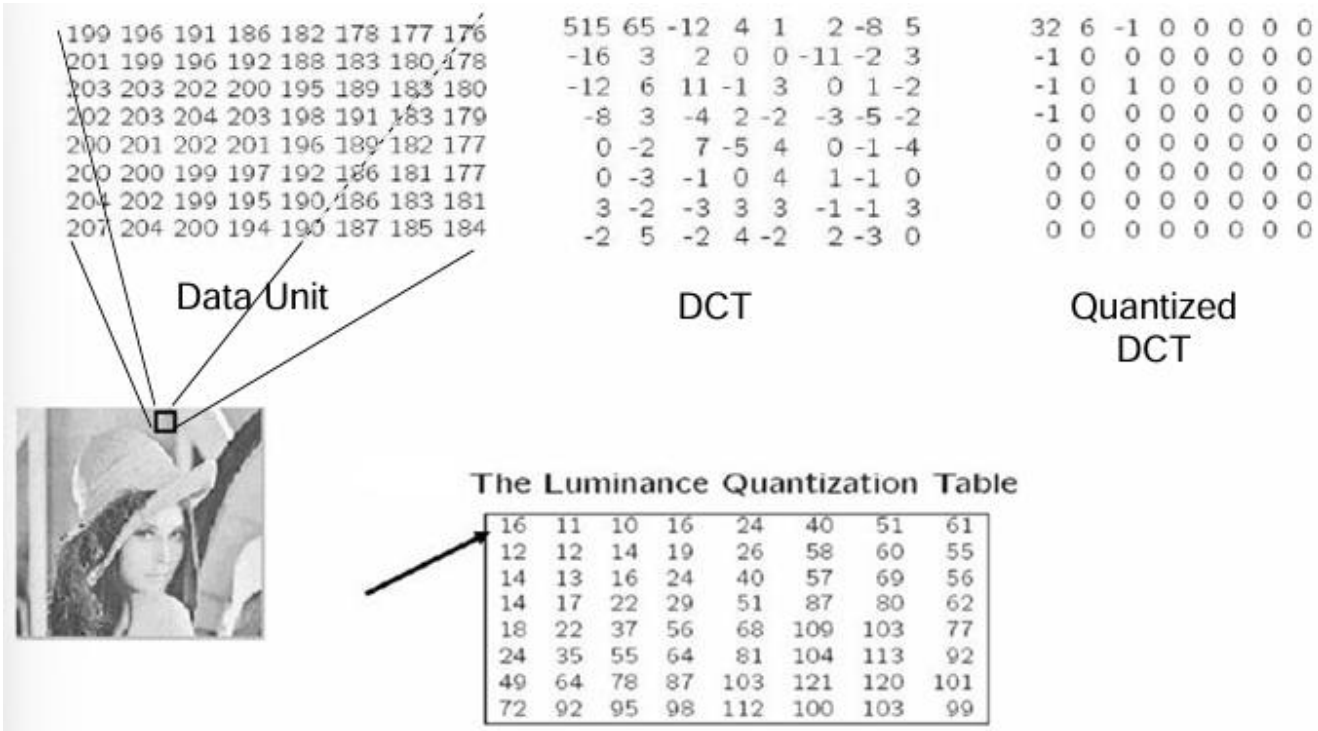
The Luminance Component Table

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

The Chrominance Component Table

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

- تتم عملية التكميم بقسمة كل معامل من معاملات DCT على القيمة المناظرة له في جدول التكميم (Element by Element) ثم يُقرب الناتج إلى أقرب عدد صحيح.
- نلاحظ أن القيم الموجودة في الزاوية العلوية اليسرى من جدول التكميم تكون صغيرة، لذلك تقسم عليها المعاملات المهمة فتتغير بدرجة بسيطة فقط. أما كلما ابتعدنا عن الزاوية، تصبح قيم جدول التكميم أكبر، ولذلك تقسم عليها المعاملات الصغيرة، وغالباً تصبح قيمها بعد التقريب مساوية للصفر، ولهذا يمكن اعتبار جدول التكميم مجموعة من العتبات التي تحدد مقدار الدقة المطلوبة لكل معامل.
- وبذلك لا يقتصر التكميم على تفسير بعض القيم، وإنما يُغيّر دقة جميع معاملات DCT بدرجات مختلفة. وعند فك الضغط تضرب القيم مرة أخرى في عناصر جدول التكميم لإعادة بناء معاملات DCT، إلا أنها لا تعود مطابقة تماماً للقيم الأصلية بسبب عملية التقريب، ولهذا تعد هذه المرحلة Lossy.
- يوضح المثال التالي خطوات تطبيق التكميم على أحد بلوكات الصورة.

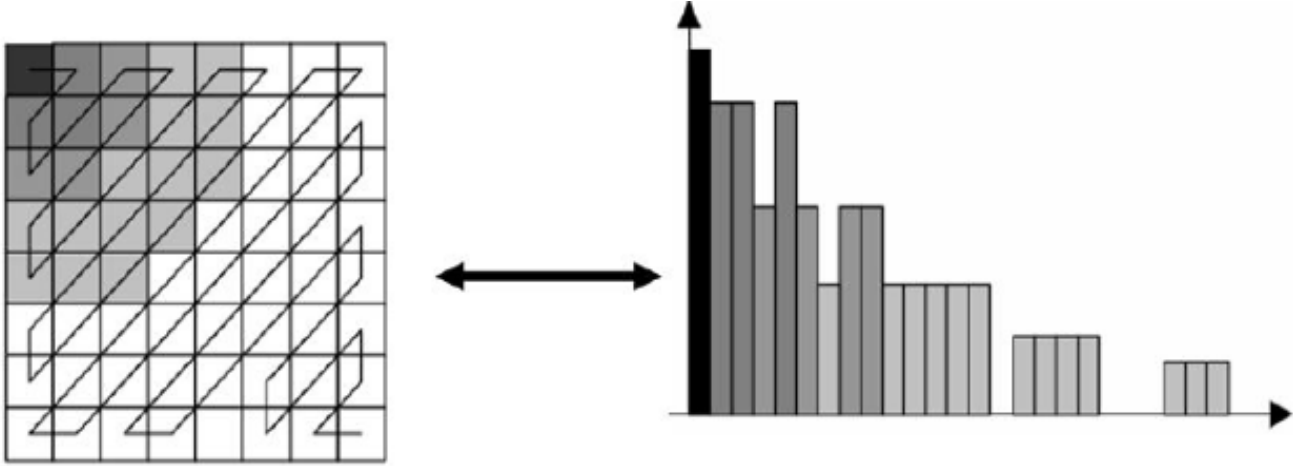


- نقوم بتطبيق تحويل DCT على بلوك 8×8، والذي يحتوي على معاملات موجبة وسالبة.
- بعد ذلك تقسم جميع المعاملات على عناصر جدول التكميم، ثم تحذف الأجزاء الكسرية، فنتج مصفوفة جديدة تحتوي على عدد كبير من الأصفار.
- نلاحظ أن معظم الأصفار تظهر في الجزء السفلي والأيمن من المصفوفة، بينما تبقى المعاملات المهمة متمركزة في الزاوية العلوية اليسرى.
- وهذه هي المصفوفة التي تُخزن في ملف JPEG، لأنها أصبحت تحتوي على عدد كبير من القيم الصفرية، مما يسهل ضغطها في المراحل اللاحقة.

ملاحظة:

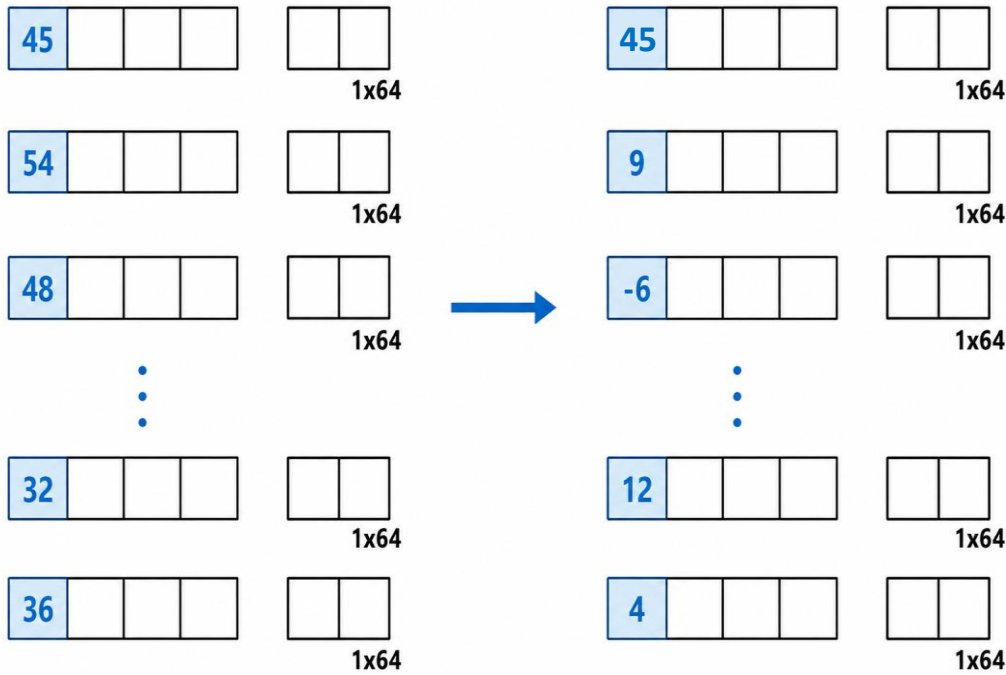
- قد يلاحظ أن المعاملات الأكبر في DCT تظهر دائماً في الزاوية العلوية اليسرى. وترجع هذه الفكرة إلى ما رأيناه سابقاً في Fourier Transform.
- في تحويل Fourier تمثل النقطة (0,0) مجموع الإشارة، ولذلك تكون قيمتها كبيرة جداً، كما أن المعاملات القريبة منها تكون غالباً أكبر من بقية المعاملات لأنها تمثل الترددات المنخفضة التي تحمل معظم طاقة الإشارة. وعند عرض نتائج فورييه تُرسم عادة قيمة لوغاريتم مقدار المعامل (Log Magnitude) حتى يمكن رؤية بقية القيم، لأن قيمة المركز تكون كبيرة جداً مقارنةً ببقيّة المصفوفة.
- أما في DCT فالفكرة نفسها موجودة؛ إذ تتركز أهم معاملات الصورة عند (u = 0, v = 0)، لكن هذه النقطة تظهر في الزاوية العلوية اليسرى بدلاً من منتصف المصفوفة، لأن DCT لا يستخدم عملية الإزاحة (Shift) كما في عرض نتائج فورييه.
- ولهذا تُحافظ جداول التكميم على المعاملات الموجودة في الزاوية، بينما تُطبق تكميماً أكبر على بقية المعاملات، لأنها أقل أهمية في تمثيل الصورة.

- بعد الانتهاء من Quantization تصبح معاملات DCT على شكل مصفوفة 8×8 ولتسهيل تخزينها ومعالجتها، لا تُخزن المصفوفة بترتيب الصفوف والأعمدة، وإنما تُقرأ وفق ترتيب قياسي يسمى Zigzag Scan.



- في هذا الأسلوب يبدأ المسار من الزاوية العلوية اليسرى، ثم يتابع وفق مسار متعرج (Zigzag) الموضح في الشكل، حتى تتحول مصفوفة 8×8 إلى متجه (Vector) مكون من 64 قيمة.
- تكمن أهمية ترتيب Zigzag في أنه يجمع المعاملات الصغيرة، والتي أصبحت في معظمها أصفاراً بعد التكميم، في نهاية السلسلة. أما لو خُزنت المصفوفة سطرًا بعد سطر، فستكون الأصفار موزعة في أماكن مختلفة ولن تتجمع معاً.
- ونتيجة لذلك تتكون سلسلة طويلة من الأصفار المتتالية، مما يجعلها مناسبة جداً لمرحلة الضغط التالية باستخدام RLE.
- كذلك تُعامل أول قيمة في السلسلة، وهي DC Coefficient، بصورة مستقلة عن بقية المعاملات AC Coefficients، لأنها تمثل أهم معامل في البلوك، بينما تكون معاملات AC أقل أهمية ويمكن ضغطها بدرجة أكبر.

- بعد تحويل كل بلوك إلى متجه باستخدام Zigzag، يصبح لدينا متجه مكون من 64 قيمة لكل بلوك، حيث تمثل أول قيمة DC Coefficient، بينما تمثل بقية القيم AC Coefficients.
- ولأن قيم DC للبلوكات المتجاورة تكون غالباً متقاربة، فإن معيار JPEG لا يرسل جميع قيم DC كما هي، وإنما يطبق عليها DPCM.



- ترسل أول قيمة DC كما هي، أما قيم DC في البلوكات التالية، فيُرسل الفرق بينها وبين قيمة DC للبلوك السابق، أي:

$$\text{Difference} = \text{DC}(\text{current}) - \text{DC}(\text{previous})$$

- وبما أن هذه الفروقات تكون أصغر من القيم الأصلية في معظم الحالات، فإنها تحتاج إلى عدد أقل من البتات عند الترميز، مما يساهم في تقليل حجم البيانات.
- أما معاملات AC فلا يُطبق عليها DPCM، وإنما تُعالج في المراحل التالية باستخدام أساليب ترميز أخرى.

8. Entropy Coding of DC Coefficients

- بعد الحصول على فروقات معاملات DC باستخدام DPCM، تأتي مرحلة Entropy Coding، حيث لا ترسل قيمة الفرق مباشرة، وإنما تُرمز بطريقة تقلل عدد البتات اللازمة لتمثيلها.
- لا ترسل قيمة الفرق وحدها، لأن المستقبل لن يعرف عدد البتات التي تمثل هذه القيمة. لذلك تقسم البيانات إلى جزأين:
- Size: يحدد عدد البتات اللازمة لتمثيل قيمة الفرق.
- Value: وهي قيمة الفرق نفسها ممثلة بهذا العدد من البتات.
- أي أن المستقبل يقرأ أولاً قيمة Size، ثم يعرف مباشرة عدد البتات التي يجب أن يقرأها بعد ذلك للحصول على قيمة الفرق.

«لا تتركّن العمر دون تعلّم
واظفر بجد وازدياد عزيمة»



SIZE	Value	Code
0	0	---
1	-1, 1	0, 1
2	-3, -2, 2, 3	00, 01, 10, 11
3	-7, ..., -4, 4, ..., 7	000, ..., 011, 100, ..., 111
4	-15, ..., -8, 8, ..., 15	0000, ..., 0111, 1000, ..., 1111
.	.	.
.	.	.
11	-2047, ..., -1024, 1024, ..., 2047	...

■ مثال:

إذا كانت قيمة الفرق تساوي 8- فإنها تنتمي إلى الفئة (Size = 4) ، لذلك تُرسل أولاً قيمة Size ، ثم تُرسل قيمة الفرق باستخدام 4 بتات، فيكون الترميز كما هو موضح:

-8 is encoded as: **1010111**

101: 4 (the size)

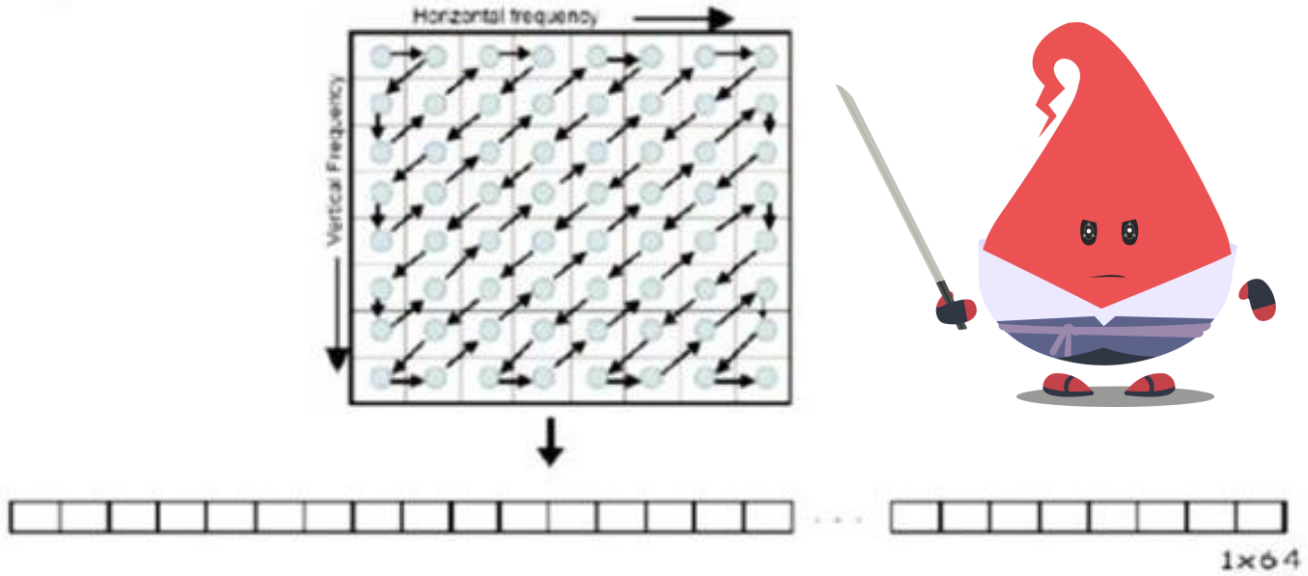
0111: -8 (The value)

■ ملاحظة:

يكون طول الحقل الذي يمثل Size ثابتاً وفق معيار JPEG، بينما يختلف عدد بتات Value بحسب قيمة الفرق، ورغم أن إرسال Size يضيف عدداً قليلاً من البتات، فإنه ضروري حتى يستطيع المستقبل معرفة كيفية قراءة البيانات وفك ترميزها بصورة صحيحة، ولو أرسلت القيمة وحدها فلن يستطيع المستقبل معرفة عدد البتات التي تمثلها، لذلك يجب إرسال قيمة Size أولاً، ثم القيمة نفسها.

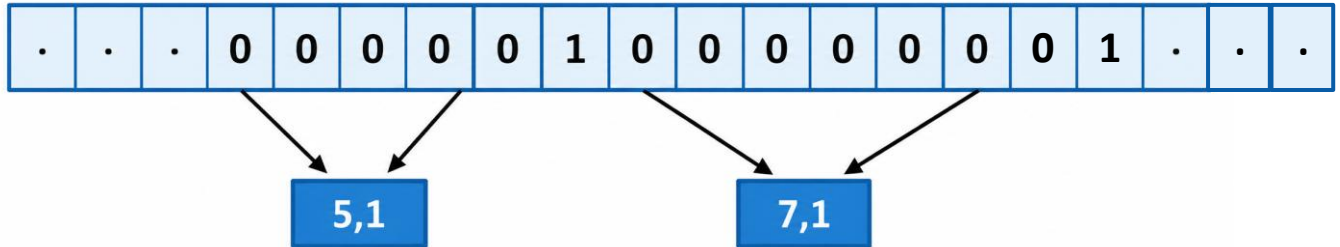
9. RLE on AC Components

■ بعد ترميز معاملات DC، تُضغط معاملات AC باستخدام RLE، وذلك بعد ترتيبها بطريقة Zig-Zag. والهدف من هذه المرحلة هو استغلال سلاسل الأصفار الطويلة التي نتجت بعد التكميم، مما يقلل كمية البيانات المطلوب تخزينها.



- بدلاً من إرسال كل قيمة على حدة، يُرسل لكل سلسلة من الأصفار زوج من القيم بالشكل (Skip, Value) كما هو موضح في الشكل التالي:
- Skip: عدد الأصفار المتتالية .
- Value: أول قيمة غير صفرية تأتي بعد هذه الأصفار .

مثال:



إذا كان هناك خمس قيم متتالية تساوي صفراً ثم جاءت القيمة 1، فإنها تُرمز بالشكل (5,1)، أي يتم تجاوز خمسة أصفار ثم كتابة القيمة 1. وإذا تبعها سبعة أصفار ثم القيمة 1 مرة أخرى، فإنها تُرمز بالشكل (7,1)، وتستمر العملية بالطريقة نفسها حتى نهاية البلوك. أما إذا انتهت بقية معاملات AC وكانت جميعها أصفاراً، فلا حاجة لإرسالها جميعاً، وإنما يُرسل الزوج (0,0) للدلالة على نهاية البلوك (End of Block - EOB).

10. Entropy Coding on RLE on AC Components

- بعد تطبيق RLE على معاملات AC، يمكن ضغط البيانات الناتجة مرة أخرى باستخدام Entropy Coding، وذلك للحصول على تمثيل أكثر كفاءة دون فقدان أي معلومات إضافية (Lossless Compression).
- في هذه المرحلة لا يُرسل الزوج (Skip, Value) مباشرة، وإنما يُمثل كل عنصر بزوج جديد هو (S1, S2) حيث يتكون:
- S1 من جزأين :

- RunLength: عدد الأصفار المتتالية .
- Size: عدد البتات اللازمة لتمثيل أول قيمة غير صفرية .
- S2 يمثل القيمة الفعلية (Value) للمعامل غير الصفري، وتُرمز باستخدام جدول Size-Value نفسه الذي استُخدم سابقاً في ترميز معاملات DC.
- وبذلك تصبح بيانات معاملات AC ممثلة بصورة أكثر كفاءة، ثم تُخزن ضمن ملف JPEG بعد الانتهاء من جميع مراحل الضغط.

يمكن تلخيص مراحل ضغط الصور في معيار JPEG كما يلي:

- تحوّل الصورة من فضاء RGB إلى فضاء يفصل مكون الإضاءة (Y) عن مكونات اللون (U و V) حتى يمكن التعامل مع كل منها بصورة مختلفة.
- تخفض دقة مكونات اللون (U و V) مع الحفاظ على دقة مكون الإضاءة (Y) ، مستفيدين من أن العين البشرية أقل حساسية لتغيرات اللون.
- تقسم الصورة إلى بلوكات مستقلة، يكون حجم كل منها عادة 8x8 بيكسل، وتعالج كل كتلة بصورة مستقلة.
- يُطبق DCT على كل بلوك لتحويله من فضاء الصورة إلى فضاء الترددات، بحيث تتركز معظم طاقة الصورة في عدد قليل من المعاملات.
- تقسم معاملات DCT على جداول التكميم ثم تقرب القيم الناتجة، مما يؤدي إلى تقليل دقة المعاملات الصغيرة أو تقريبها إلى الصفر، وهي المرحلة التي يحدث فيها الفقد. (Lossy Compression)
- ترتب معاملات DCT بطريقة Zigzag لتجميع المعاملات المهمة في البداية، وسلاسل الأصفار في النهاية، مما يسهل ضغطها.
- ترمز معاملات DC باستخدام DPCM ، وذلك بإرسال الفروقات بين معاملات DC للبلوكات المتتالية بدلاً من إرسال قيمها مباشرة.
- تطبق إحدى تقنيات Entropy Coding على بيانات DC للحصول على تمثيل أكثر كفاءة دون فقدان أي معلومات.
- ترمز معاملات AC باستخدام (RLE)، وذلك بتمثيل سلاسل الأصفار المتتالية بعدد مرات تكرارها بدلاً من إرسال كل صفر على حدة.
- تطبق إحدى تقنيات Entropy Coding على البيانات الناتجة من RLE للحصول على ضغط إضافي دون فقدان أي معلومات قبل تخزين الملف.
- تتم عملية فك الضغط بتنفيذ جميع الخطوات السابقة بالترتيب العكسي، بدءاً من فك ترميز البيانات، ثم استعادة معاملات DCT، وتطبيق التحويل العكسي IDCT، وصولاً إلى إعادة الصورة إلى فضاء RGB.
- لا يعتمد JPEG على خوارزمية واحدة، وإنما على مجموعة من تقنيات الضغط التي تعمل معاً. ويُعد تحويل الصورة إلى فضاء DCT ثم إجراء التكميم العامل الأساسي الذي يحقق نسب الضغط العالية، بينما تسهم بقية المراحل، مثل تقليل دقة مكونات اللون، وDPCM، وRLE، وEntropy Coding، في زيادة كفاءة الضغط دون التأثير الكبير في جودة الصورة.

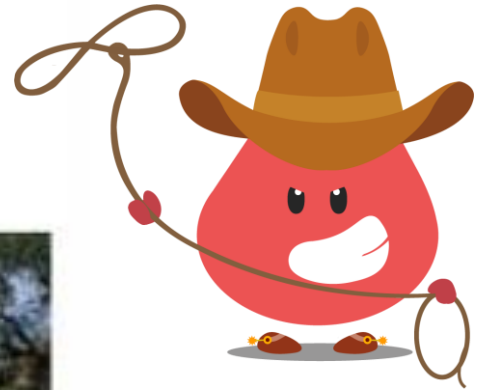
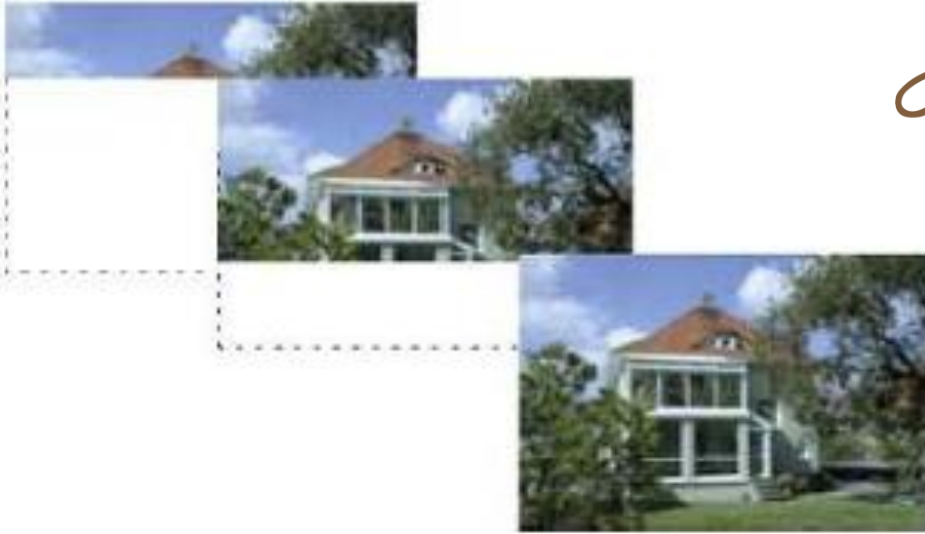
JPEG Modes

بعد الانتهاء من جميع مراحل ضغط الصورة، يوفر معيار JPEG عدة أنماط (Modes) لتخزين البيانات داخل الملف. ولا يختلف كل نمط في آلية الضغط نفسها، وإنما يختلف في طريقة ترتيب وإرسال البيانات، وهو ما يؤثر في كيفية ظهور الصورة أثناء فك الضغط، وخاصة عند نقلها عبر الشبكات.

Sequential Mode

يُعد Sequential Mode النمط الأبسط والأكثر شيوعاً في JPEG، حيث تُضغط الصورة كاملة ثم تُخزن وتُفك بعملية واحدة، ويكون ترتيب البيانات داخل الملف وفق مسح متتابع (Scan) يبدأ من أعلى الصورة إلى أسفلها سطرًا بعد سطر. وعند عرض الصورة من ملف مخزن على الحاسب لا يلاحظ المستخدم هذا الترتيب، لأن البرنامج يفك الملف كاملاً قبل عرضه. أما عند إرسال الصورة عبر الإنترنت، فتصل البيانات تدريجياً، لذلك قد تظهر الصورة وهي تُحمّل سطرًا بعد سطر حتى تكتمل بالكامل.

تعد هذه الطريقة مناسبة في كثير من التطبيقات، إلا أنها قد لا تكون الأفضل عند بطء الاتصال؛ إذ قد يضطر المستخدم إلى انتظار تحميل جزء كبير من الصورة قبل أن يتمكن من معرفة محتواها.



Progressive Mode

في Progressive Mode لا تُرسل بيانات الصورة دفعة واحدة، وإنما تُرسل على عدة مراحل (Multiple Scans)، بحيث يستطيع المستقبل عرض الصورة تدريجياً مع استمرار وصول البيانات. لذلك تظهر الصورة في البداية بجودة منخفضة أو بصورة ضبابية، ثم تتحسن تدريجياً مع وصول المزيد من البيانات، إلى أن تصل في النهاية إلى جودتها الكاملة. ويُعد هذا النمط مناسباً لإرسال الصور عبر الشبكات، لأنه يسمح للمستخدم برؤية محتوى الصورة بسرعة دون الحاجة إلى انتظار اكتمال تحميلها بالكامل. فإذا تبين أن الصورة ليست المطلوبة، يمكنه إيقاف التحميل والانتقال إلى صورة أخرى قبل انتهاء عملية النقل.

ويتحقق هذا الأسلوب بطريقتين أساسيتين هما Spectral Selection و Successive Approximation، واللذان يختلفان في كيفية إرسال بيانات الصورة، مع الحفاظ على النتيجة النهائية نفسها بعد اكتمال الاستقبال.



1. Spectral Selection

في أسلوب Spectral Selection لا ترسل معاملات DCT دفعةً واحدة، وإنما ترسل على مراحل بحسب أهميتها.

فبدلاً من إرسال جميع معاملات البلوك الأول ثم الانتقال إلى البلوك الثاني، ترسل أولاً معاملات DC لجميع البلوكات، ثم ترسل أول معاملات AC لجميع البلوكات، ثم المجموعة التالية من معاملات AC ، وهكذا حتى تكتمل جميع معاملات DCT .

ويعني ذلك أنه بمجرد وصول معاملات DC يستطيع المستقبل وضع قيمة هذه المعاملات، وافترض أن بقية معاملات AC تساوي صفراً، ثم يطبق التحويل العكسي IDCT ليحصل على صورة أولية منخفضة الجودة. وبعد وصول المجموعة التالية من معاملات AC يعيد تنفيذ IDCT مرة أخرى فتنحسن الصورة، وتكرر العملية مع كل مجموعة جديدة من المعاملات حتى تكتمل الصورة.

وبذلك تظهر الصورة كاملة منذ البداية ولكن بصورة ضبابية، ثم تتحسن تدريجياً مع وصول معاملات جديدة. ويتميز هذا الأسلوب عند عرض الصور عبر الانترنت، لأن المستخدم يستطيع تكوين فكرة عن محتوى الصورة بسرعة، فإذا لم تكن الصورة المطلوبة يمكنه إيقاف التحميل دون انتظار اكتماله.

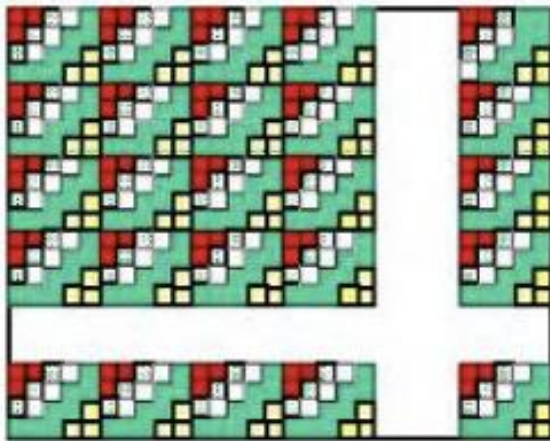


Image Pixels

Spectral Selection:

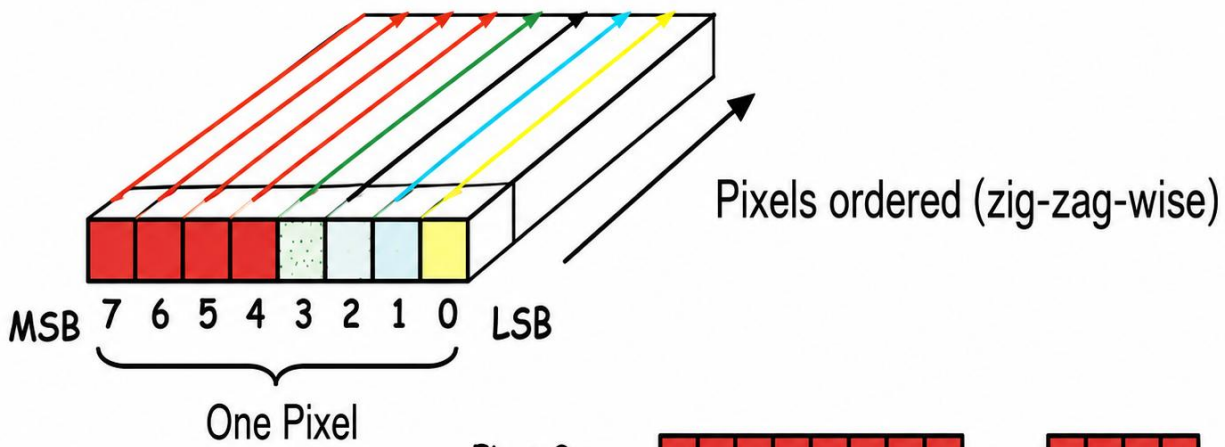
First Scan:	
Second Scan:	
Third Scan:	
N th Scan:	

2. Successive Approximation

أما في Successive Approximation فلا يُقسَّم الإرسال بحسب نوع معاملات DCT ، وإنما بحسب عدد البتات (Bits) التي تمثل كل معامل.

فبدلاً من إرسال قيمة كل معامل كاملة مرة واحدة، تُرسل في البداية البتات الأكثر أهمية MSB لجميع المعاملات، بينما تُفترض البتات غير المرسلَة أصفاراً، ثم يُطبق التحويل العكسي للحصول على صورة أولية منخفضة الجودة.

بعد ذلك تُرسل دفعة جديدة من البتات الأقل أهمية LSB لجميع المعاملات، فتُستكمل قيمها بصورة أدق، ثم يُعاد تنفيذ التحويل العكسي مرة أخرى، فتزداد جودة الصورة. وتكرر هذه العملية حتى تُرسل جميع البتات، فتصل الصورة إلى جودتها النهائية.



First Scan: [Red blocks] ... [Red blocks]

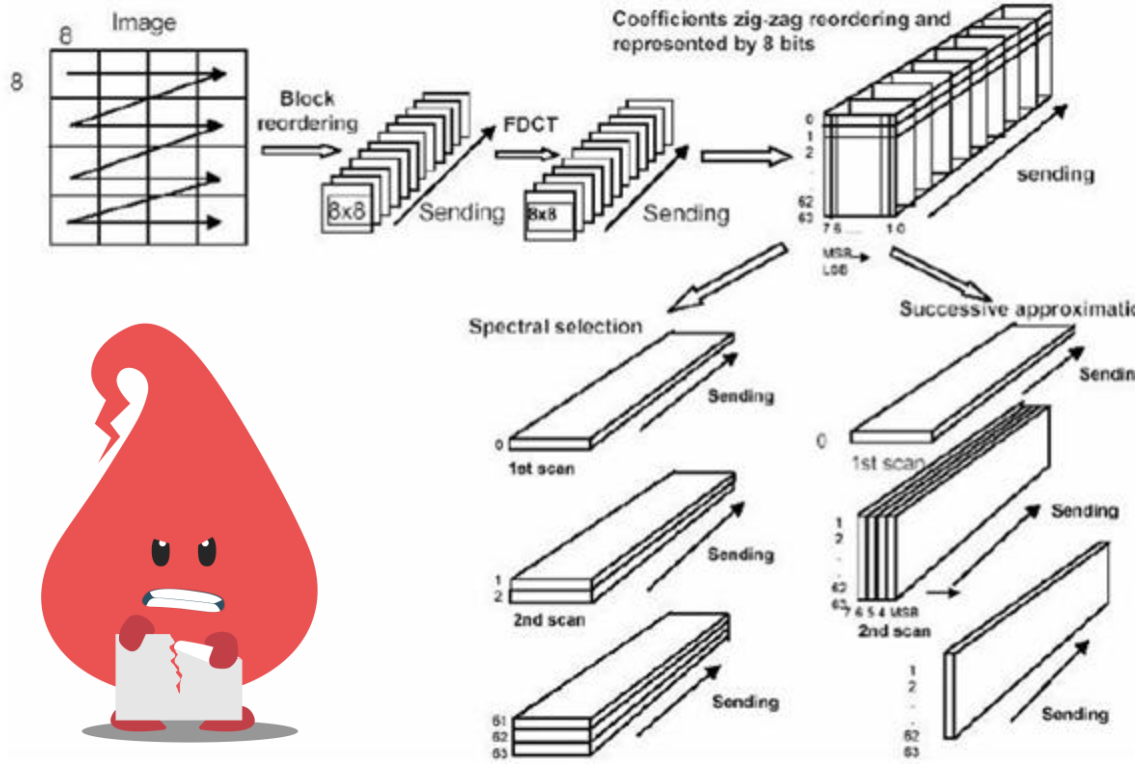
Second Scan: [Green blocks] ... [Green blocks]

Third Scan: [White blocks] ... [White blocks]

⋮

5th Scan: [Yellow blocks] ... [Yellow blocks]

يكمن الفرق الجوهرى بين الطريقتين في أن أسلوب Spectral Selection يحسن الصورة بإرسال معاملات DCT جديدة في كل مرحلة (DC ثم AC ثم AC...)، بينما يحسن Successive Approximation الصورة بإرسال بتات جديدة للمعاملات نفسها، فتزداد دقتها تدريجياً مع كل مرحلة حتى تكتمل.



Hierarchical Mode

يُعد Hierarchical Mode نمطاً آخر من أنماط JPEG، ويعتمد على تمثيل الصورة بعدة مستويات من الدقة.

في البداية تخفض دقة الصورة (Down Sampling) للحصول على نسخة منخفضة الدقة، ثم تضغط وترسل هذه النسخة أولاً. بعد ذلك ترسل بيانات إضافية تُستخدم لإعادة بناء الصورة بدقة أعلى تدريجياً، إلى أن تصل إلى الدقة الأصلية.

لذلك يستطيع المستقبل عرض نسخة منخفضة الدقة بسرعة، ثم تحسينها تدريجياً مع وصول المزيد من البيانات، وهو ما يجعله مناسباً للتطبيقات التي تحتاج إلى عرض الصورة بأكثر من مستوى دقة.

